

SKRIPSI

**PERANGKAT LUNAK PEMBUAT VISUALISASI GRAF SITUS
WEB**



Bram Matthew

NPM: 6182101043

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS
UNIVERSITAS KATOLIK PARAHYANGAN
2026**

DAFTAR ISI

DAFTAR ISI	iii
DAFTAR GAMBAR	v
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan	2
1.4 Batasan Masalah	2
1.5 Metodologi	2
1.6 Sistematika Pembahasan	2
2 LANDASAN TEORI	3
2.1 Hypertext Markup Language (HTML)	3
2.1.1 Pengertian HTML	3
2.1.2 Elemen dan Tag HTML	3
2.1.3 Struktur dasar HTML	4
2.1.4 Hierarki elemen HTML	5
2.2 Document Object Model (DOM)	6
2.2.1 Pengertian DOM	6
2.2.2 Struktur DOM sebagai Tree	6
2.2.3 Jenis node pada DOM	6
2.3 Parsing HTML	6
2.3.1 Pengertian Parsing HTML	6
2.3.2 Hubungan Parsing HTML dengan DOM	7
2.4 Struktur Data Tree	7
2.4.1 Pengertian Tree	7
2.4.2 Tree pada Struktur DOM	7
2.5 Struktur Data Graf	7
2.5.1 Pengertian Graf	7
2.5.2 Representasi Graf	8
2.5.3 Graf Berarah	8
2.5.4 Representasi DOM sebagai graf	9
2.5.5 Pengertian Traversal Graf	10
2.5.6 Depth-First Search	10
2.6 Graphviz	11
2.6.1 Pengertian Graphviz	11
2.6.2 Format DOT	12
DAFTAR REFERENSI	13
A KODE PROGRAM	15

DAFTAR GAMBAR

2.1	Gambar Tag pembuka dan penutup	4
2.2	Gambar Struktur dasar HTML	5
2.3	Contoh graf berarah sederhana (node dan edge)	9
2.4	Ilustrasi proses DFS pada Graf berarah	11
2.5	Potongan Kode dengan format DOT	12
2.6	Visualisasi dari format DOT dengan menggunakan Graphviz	12
B.1	Hasil 1	17
B.2	Hasil 2	17
B.3	Hasil 3	17
B.4	Hasil 4	17

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi web yang sangat pesat menyebabkan halaman web modern memiliki struktur yang semakin kompleks. Setiap halaman web dibangun menggunakan *Hypertext Markup Language* (HTML) yang secara umum dapat direpresentasikan dalam bentuk struktur hierarkis yang dikenal sebagai *Document Object Model* (DOM). Struktur DOM ini berbentuk pohon (*tree*) yang terdiri dari elemen-elemen HTML yang saling terhubung melalui hubungan induk (*parent*) dan anak (*child*)^[1].

Meskipun struktur DOM merupakan konsep dasar dalam pengembangan web, representasinya dalam bentuk kode HTML sering kali sulit dipahami secara langsung, terutama pada halaman web yang memiliki banyak elemen. Oleh karena itu, diperlukan suatu cara yang dapat membantu menyajikan struktur tersebut dalam bentuk yang lebih mudah dipahami.

Salah satu cara yang dapat digunakan adalah dengan memvisualisasikan struktur halaman web dalam bentuk graf. Dalam pendekatan ini, setiap elemen HTML direpresentasikan sebagai simpul (*node*) dan hubungan antar elemen direpresentasikan sebagai sisi (*edge*). Dengan demikian, struktur DOM dapat dilihat secara lebih jelas dan terstruktur.

Berdasarkan permasalahan tersebut, tugas akhir ini berfokus pada pengembangan perangkat lunak yang mampu mengambil dokumen HTML dari suatu *Uniform Resource Locator* (URL), membangun struktur DOM, dan merepresentasikannya dalam bentuk graf yang dapat divisualisasikan. Perangkat lunak ini diharapkan dapat membantu pengguna dalam memahami struktur halaman web secara lebih mudah.

1.2 Rumusan Masalah

Rumusan Masalah yang akan dibahas pada tugas akhir ini adalah:

1. Bagaimana cara mengambil dokumen HTML dari suatu situs web berdasarkan URL serta membangun struktur DOM?
2. Bagaimana cara merepresentasikan struktur DOM ke dalam bentuk graf yang terdiri dari (*node*) dan (*edge*) melalui proses traversal yang sistematis?
3. Bagaimana merancang perangkat lunak yang mampu menghasilkan visualisasi graf dari struktur halaman web secara otomatis dan mudah dipahami pengguna?

1.3 Tujuan

Tujuan yang ingin dicapai skripsi ini adalah sebagai berikut:

1. Mengambil dokumen HTML dari suatu situs web berdasarkan URL serta membangun struktur DOM.
2. Merepresentasikan struktur DOM ke dalam bentuk graf yang terdiri dari (*node*) dan (*edge*) melalui proses traversal yang sistematis.
3. Merancang perangkat lunak yang mampu menghasilkan visualisasi graf dari struktur halaman web secara otomatis dan mudah dipahami pengguna.

1.4 Batasan Masalah

Pada pengerjaan tugas akhir ini terhadap batasan sebagai berikut:

- Perangkat lunak dikembangkan menggunakan bahasa pemrograman Java

1.5 Metodologi

Metodologi pengerjaan tugas akhir ini adalah sebagai berikut:

1. Melakukan studi literatur mengenai HTML, DOM, struktur data graf, Graphviz, dan metode traversal.
2. Melakukan analisis kebutuhan serta perancangan arsitektur perangkat lunak yang akan dikembangkan.
3. Mengimplementasikan sistem untuk mengambil dokumen HTML, membangun struktur DOM, serta merepresentasikannya dalam bentuk graf.
4. Melakukan pengujian dan eksperimen menggunakan beberapa halaman web dengan struktur yang berbeda.
5. Melakukan analisis terhadap hasil visualisasi yang dihasilkan oleh sistem.
6. Menyusun dokumentasi dan laporan tugas akhir.

1.6 Sistematika Pembahasan

Sistematika pembahasan skripsi ini adalah sebagai berikut:

- Bab 1: Pendahuluan Membahas latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan.
- Bab 2: Landasan Teori Membahas dasar dan hierarki HTML, struktur dan jenis node DOM, Graf berarah, DOT file dan Graphviz.
- Bab 3: Analisis
- Bab 4: Perancangan
- Bab 5: Implementasi dan Pengujian
- Bab 6: Kesimpulan dan Saran

BAB 2

LANDASAN TEORI

2.1 Hypertext Markup Language (HTML)

2.1.1 Pengertian HTML

Hypertext Markup Language (HTML) merupakan bahasa markup yang digunakan untuk menyusun struktur dasar halaman web. HTML tidak termasuk bahasa pemrograman karena tidak memiliki mekanisme logika seperti percabangan, perulangan, atau pengolahan data secara langsung, melainkan digunakan untuk mendeskripsikan struktur dan makna konten yang akan ditampilkan pada browser [2]. HTML memungkinkan pengembang mengatur berbagai jenis konten, seperti teks, gambar, tautan, tabel, formulir, serta elemen multimedia lainnya dalam sebuah dokumen web.

HTML berfungsi sebagai kerangka utama halaman web yang menentukan bagaimana setiap bagian konten disusun sehingga dapat ditampilkan secara terstruktur oleh browser [2]. Dalam pengembangan web, HTML umumnya digunakan bersama dengan CSS dan JavaScript. HTML berperan dalam membentuk struktur konten, CSS digunakan untuk mengatur tampilan atau gaya visual, sedangkan JavaScript digunakan untuk menambahkan interaktivitas pada halaman web.

HTML bekerja dengan konsep *tag-based structure*, yaitu setiap elemen ditandai dengan tag tertentu. Tag tersebut memberikan informasi kepada browser mengenai jenis dan fungsi konten, misalnya paragraf, judul, gambar, tautan, atau daftar. Melalui struktur ini, browser dapat memahami peran masing-masing elemen, menyusun hubungan antar elemen, dan menampilkannya dengan benar kepada pengguna.

2.1.2 Elemen dan Tag HTML

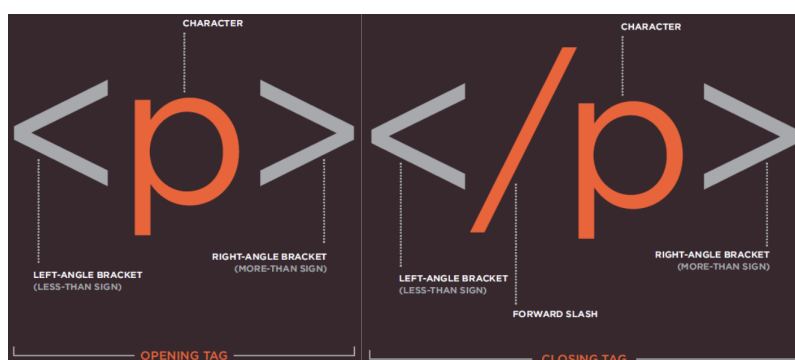
Elemen HTML merupakan komponen dasar dalam penyusunan halaman web. Setiap elemen digunakan untuk merepresentasikan bagian tertentu dari konten, seperti paragraf, judul, gambar, tautan, tabel, daftar, maupun formulir. Secara umum, sebuah elemen HTML terdiri dari tag pembuka, isi elemen, dan tag penutup. Sebagai contoh, elemen paragraf dituliskan dengan <p> sebagai tag pembuka, isi teks sebagai konten, dan </p> sebagai tag penutup.

Tag berfungsi sebagai penanda yang memberi tahu browser mengenai jenis elemen yang sedang digunakan. Tag pembuka menunjukkan awal suatu elemen, sedangkan tag penutup menunjukkan akhir elemen tersebut. Namun, tidak semua elemen HTML memiliki tag penutup. Beberapa elemen, seperti ,
, dan <hr>, termasuk elemen kosong karena tidak memiliki isi di antara tag pembuka dan tag penutup.

Selain tag, elemen HTML juga dapat memiliki atribut yang memberikan informasi tambahan

terhadap elemen tersebut. Atribut dituliskan di dalam tag pembuka dan umumnya terdiri dari nama atribut serta nilai atribut. Sebagai contoh, atribut href pada elemen `<a>` digunakan untuk menentukan alamat tautan, sedangkan atribut src pada elemen `<img>` digunakan untuk menentukan lokasi berkas gambar. Atribut lain seperti id dan class sering digunakan untuk memberi identitas atau pengelompokan elemen agar dapat diatur lebih lanjut menggunakan CSS atau diproses melalui JavaScript.

Struktur elemen yang saling bersarang (*nested elements*) memungkinkan terbentuknya hubungan hierarkis antar elemen. Dalam struktur ini, suatu elemen dapat menjadi induk bagi elemen lain yang berada di dalamnya. Misalnya, elemen `<body>` dapat memuat elemen `<div>`, kemudian elemen `<div>` tersebut dapat memuat elemen `<p>`, `<img>`, atau `<a>`. Hubungan bertingkat ini menjadi dasar penting dalam pembentukan struktur DOM karena browser akan membaca dokumen HTML sebagai susunan node yang saling terhubung.



Gambar 2.1: Contoh tag pembuka dan penutup dokumen HTML

2.1.3 Struktur dasar HTML

Dokumen HTML memiliki struktur standar yang terdiri dari beberapa bagian utama, yaitu `<html>`, `<head>`, dan `<body>`. Struktur tersebut biasanya diawali dengan deklarasi `<!DOCTYPE html>` yang berfungsi untuk memberi tahu browser bahwa dokumen menggunakan standar HTML modern. Setelah deklarasi tersebut, elemen `<html>` digunakan sebagai root dari seluruh dokumen HTML dan menjadi wadah bagi elemen-elemen lain di dalam halaman web.

Elemen `<head>` berisi informasi yang tidak ditampilkan secara langsung pada halaman, tetapi dibutuhkan oleh browser dan mesin pencari untuk memahami dokumen. Informasi tersebut dapat berupa judul halaman melalui elemen `<title>`, pengaturan karakter melalui elemen `<meta>`, pemanggilan berkas CSS melalui elemen `<link>`, serta pemanggilan skrip tertentu melalui elemen `<script>`. Bagian ini berperan penting dalam pengaturan metadata, optimasi halaman, dan hubungan dokumen HTML dengan sumber daya eksternal.

Elemen `<body>` berisi seluruh konten utama yang ditampilkan kepada pengguna pada browser. Konten tersebut dapat berupa teks, gambar, tautan, tabel, formulir, daftar, maupun elemen multimedia. Dengan demikian, bagian `<body>` menjadi tempat penyusunan tampilan utama halaman web yang dapat dilihat dan berinteraksi langsung dengan pengguna.

Struktur dasar ini memastikan bahwa setiap halaman web memiliki susunan yang konsisten dan dapat dipahami oleh browser. Ketika sebuah dokumen HTML dimuat, browser akan membaca struktur tersebut dari bagian paling atas, memproses metadata pada `<head>`, kemudian

- 1 menampilkan konten yang terdapat pada `<body>`. Susunan yang teratur juga memudahkan proses
- 2 pembentukan DOM karena setiap elemen memiliki posisi dan hubungan yang jelas di dalam hierarki
- 3 dokumen.

```
<html>
  <body>
    <h1>This is the Main Heading</h1>
    <p>This text might be an introduction to the rest of
      the page. And if the page is a long one it might
      be split up into several sub-headings.</p>
    <h2>This is a Sub-Heading</h2>
    <p>Many long articles have sub-headings so to help
      you follow the structure of what is being written.
      There may even be sub-sub-headings (or lower-level
      headings).</p>
    <h2>Another Sub-Heading</h2>
    <p>Here you can see another sub-heading.</p>
  </body>
</html>
```

Gambar 2.2: Contoh struktur dasar dokumen HTML

4 2.1.4 Hierarki elemen HTML

- 5 Struktur HTML bersifat hierarkis, yang berarti elemen dapat berada di dalam elemen lain. Hubungan
- 6 antar elemen ini membentuk struktur bertingkat yang menyerupai pohon, di mana setiap elemen
- 7 memiliki posisi tertentu dalam dokumen. Dalam struktur tersebut, elemen yang memuat elemen
- 8 lain disebut sebagai parent, sedangkan elemen yang berada di dalamnya disebut sebagai child.
- 9 Elemen-elemen yang berada pada tingkat yang sama dan memiliki parent yang sama disebut sebagai
- 10 sibling.

- 11 Sebagai contoh, elemen `<body>` dapat berperan sebagai parent bagi elemen `<div>`. Selanjutnya,
- 12 elemen `<div>` dapat berisi elemen `<p>`, `<img>`, atau `<a>` sebagai child. Jika di dalam satu
- 13 elemen `<div>` terdapat beberapa elemen `<p>`, maka elemen-elemen tersebut dapat disebut sebagai
- 14 sibling karena berada pada tingkat hierarki yang sama. Hubungan seperti ini menunjukkan bahwa
- 15 dokumen HTML tidak hanya tersusun secara berurutan, tetapi juga memiliki hubungan induk-anak
- 16 yang jelas antar elemen.

- 17 Hierarki ini penting karena menentukan bagaimana elemen saling berhubungan dan bagaimana
- 18 struktur tersebut akan diproses menjadi DOM. Ketika browser membaca dokumen HTML, setiap
- 19 elemen akan direpresentasikan sebagai node dalam struktur DOM. Hubungan parent, child, dan
- 20 sibling pada HTML akan menjadi dasar pembentukan hubungan antar node di dalam DOM. Oleh
- 21 karena itu, susunan elemen yang benar akan memudahkan browser dalam memahami struktur
- 22 dokumen dan menampilkan halaman web secara sesuai.

- 23 Semakin kompleks halaman web, maka semakin dalam dan luas struktur hierarki yang terbentuk.
- 24 Halaman yang memiliki banyak bagian, seperti header, navigasi, konten utama, sidebar, formulir, dan
- 25 footer, akan menghasilkan struktur HTML yang lebih bercabang. Struktur hierarkis tersebut juga
- 26 memengaruhi proses penelusuran elemen, baik dalam manipulasi DOM menggunakan JavaScript
- 27 maupun dalam analisis struktur halaman menggunakan pendekatan traversal seperti DFS.

2.2 Document Object Model (DOM)

2.2.1 Pengertian DOM

Document Object Model (DOM) adalah representasi dari dokumen HTML dalam bentuk struktur objek yang dapat diakses dan dimanipulasi oleh program. DOM memungkinkan setiap elemen dalam halaman web diperlakukan sebagai objek yang memiliki properti dan metode [1].

Menurut *World Wide Web Consortium*, DOM menyediakan antarmuka standar untuk mengakses dan memodifikasi dokumen HTML.

Dengan adanya DOM, halaman web tidak hanya menjadi tampilan statis, tetapi juga dapat diproses dan dimodifikasi secara dinamis.

2.2.2 Struktur DOM sebagai Tree

Struktur DOM berbentuk pohon (tree structure), di mana setiap elemen HTML direpresentasikan sebagai node. Node paling atas disebut root node, yang biasanya adalah elemen `<html>`. Setiap node dapat memiliki:

- Parent (induk)
- Child (anak)
- Sibling (saudara)

Struktur ini memungkinkan proses traversal dilakukan secara sistematis, seperti menggunakan DFS.

2.2.3 Jenis node pada DOM

DOM terdiri dari beberapa jenis node, antara lain:

- Element node: merepresentasikan tag HTML
- Text node: merepresentasikan isi teks
- Attribute node: merepresentasikan atribut

Keberadaan berbagai jenis node ini membuat struktur DOM lebih kompleks dibandingkan HTML mentah, sehingga analisis DOM memberikan informasi yang lebih lengkap.

2.3 Parsing HTML

2.3.1 Pengertian Parsing HTML

Parsing HTML adalah proses membaca dan menganalisis dokumen HTML untuk membentuk struktur yang dapat dipahami dan diproses oleh program. Pada proses ini, teks HTML yang tersusun dari tag, atribut, dan konten akan diinterpretasikan menjadi struktur yang lebih terorganisasi. Browser melakukan proses parsing untuk memahami susunan dokumen HTML sebelum membentuk DOM dan menampilkan halaman web kepada pengguna.

Dalam konteks pengolahan dokumen web, parsing HTML penting karena dokumen HTML tidak hanya berisi teks biasa, tetapi juga memiliki struktur tag yang saling bersarang. Melalui proses parsing, setiap elemen dapat dikenali berdasarkan tag, atribut, posisi, dan hubungannya dengan elemen lain. Hasil parsing inilah yang kemudian dapat digunakan untuk analisis struktur halaman, ekstraksi informasi, maupun pembentukan representasi DOM.

2.3.2 Hubungan Parsing HTML dengan DOM

Parsing HTML memiliki hubungan erat dengan pembentukan DOM. Ketika sebuah dokumen HTML diproses, parser akan membaca elemen-elemen HTML secara berurutan dan membentuk struktur node yang merepresentasikan dokumen tersebut. Struktur node ini kemudian menjadi DOM tree, yaitu representasi hierarkis dari dokumen HTML.

Pada penelitian ini, proses parsing HTML menjadi tahap awal yang penting sebelum struktur DOM direpresentasikan sebagai graf. Elemen-elemen hasil parsing dapat dipetakan menjadi node, sedangkan hubungan antar elemen seperti parent dan child dapat dipetakan menjadi edge. Dengan demikian, parsing HTML menjadi penghubung antara dokumen HTML mentah dan struktur graf yang dapat dianalisis serta divisualisasikan.

2.4 Struktur Data Tree

2.4.1 Pengertian Tree

Tree adalah struktur data hierarkis yang terdiri dari sekumpulan node dan hubungan antar node. Node paling atas dalam tree disebut root, sedangkan node yang berada di bawah node lain disebut child. Node yang memiliki child disebut parent, sementara node yang tidak memiliki child disebut leaf. Struktur tree digunakan untuk merepresentasikan hubungan bertingkat, yaitu satu node dapat memiliki beberapa node turunan.

Secara konsep, tree merupakan bentuk khusus dari graf yang tidak memiliki siklus dan memiliki hubungan hierarkis yang jelas. Setiap node dalam tree, kecuali root, memiliki satu parent. Karakteristik ini membuat tree cocok digunakan untuk memodelkan struktur yang tersusun secara bertingkat, seperti struktur folder, organisasi data, ekspresi matematika, dan dokumen HTML.

2.4.2 Tree pada Struktur DOM

DOM memiliki karakteristik yang menyerupai tree karena setiap elemen dalam dokumen HTML tersusun secara hierarkis. Elemen `<html>` biasanya menjadi root, sedangkan elemen `<head>` dan `<body>` menjadi child dari elemen tersebut. Selanjutnya, elemen-elemen lain seperti `<div>`, `<p>`, `<img>`, dan `<a>` dapat menjadi turunan dari elemen `<body>` atau elemen lain di dalamnya.

Pemahaman mengenai tree penting dalam penelitian ini karena struktur DOM yang berbentuk tree dapat ditelusuri menggunakan algoritma traversal. Salah satu algoritma yang sesuai untuk struktur seperti ini adalah DFS, karena DFS menelusuri node secara mendalam dari root menuju leaf sebelum kembali ke cabang lain. Dengan demikian, konsep tree menjadi dasar untuk memahami hubungan antara DOM, graf, dan proses traversal.

2.5 Struktur Data Graf

2.5.1 Pengertian Graf

Graf adalah struktur data yang terdiri dari simpul (*node* atau *vertex*) dan sisi (*edge*) yang menghubungkan simpul-simpul tersebut. Dalam kajian algoritma, graf umumnya dinyatakan sebagai

pasangan himpunan $G = (V, E)$, dengan V sebagai himpunan simpul dan E sebagai himpunan sisi yang merepresentasikan hubungan antar simpul [3]. Setiap simpul dapat menggambarkan suatu objek, sedangkan sisi menunjukkan adanya relasi atau keterhubungan antara dua objek.

Menurut Cormen dkk., graf banyak digunakan untuk memodelkan berbagai jenis hubungan dalam komputasi karena bentuknya fleksibel dan dapat merepresentasikan struktur yang sederhana maupun kompleks [3]. Sebagai contoh, simpul dapat merepresentasikan kota, halaman web, pengguna, proses, atau elemen dalam dokumen, sedangkan sisi dapat merepresentasikan jalan, tautan, hubungan sosial, komunikasi antar proses, atau hubungan induk-anak antar elemen.

Graf dapat dibedakan berdasarkan karakteristik sisinya. Pada graf tidak berarah, hubungan antar simpul tidak memiliki arah tertentu sehingga relasi antara dua simpul dianggap berlaku dua arah. Sebaliknya, pada graf berarah, setiap sisi memiliki arah dari satu simpul ke simpul lainnya. Selain itu, graf juga dapat memiliki bobot pada sisi untuk menyatakan nilai tertentu, seperti jarak, biaya, waktu, atau tingkat hubungan antar simpul.

Dalam konteks penelitian ini, konsep graf digunakan untuk merepresentasikan struktur DOM dari dokumen HTML. Setiap elemen HTML dapat dipandang sebagai simpul, sedangkan hubungan antara elemen induk dan elemen anak dapat dipandang sebagai sisi. Dengan pemodelan tersebut, struktur halaman web dapat dianalisis menggunakan konsep teori graf, seperti hubungan antar node, arah hubungan, dan proses traversal.

2.5.2 Representasi Graf

Representasi graf adalah cara menyimpan struktur graf agar dapat diproses oleh komputer. Dalam implementasi algoritma, graf tidak cukup hanya didefinisikan sebagai himpunan simpul dan sisi, tetapi perlu direpresentasikan dalam bentuk data tertentu. Dua bentuk representasi graf yang umum digunakan adalah adjacency matrix dan adjacency list [3].

Adjacency matrix merepresentasikan graf dalam bentuk matriks dua dimensi. Setiap baris dan kolom mewakili simpul, sedangkan nilai pada sel matriks menunjukkan ada atau tidaknya sisi antara dua simpul. Representasi ini mudah dipahami dan efisien untuk memeriksa hubungan langsung antar simpul, tetapi dapat menggunakan ruang penyimpanan yang besar jika jumlah simpul banyak dan jumlah sisi sedikit.

Adjacency list merepresentasikan graf dengan menyimpan daftar tetangga untuk setiap simpul. Setiap simpul memiliki daftar node lain yang terhubung dengannya. Representasi ini lebih hemat ruang untuk graf yang memiliki jumlah sisi relatif sedikit dibandingkan jumlah kemungkinan sisi. Dalam konteks DOM, adjacency list sesuai digunakan karena setiap elemen biasanya hanya memiliki hubungan langsung dengan sejumlah child tertentu, bukan dengan seluruh elemen dalam dokumen.

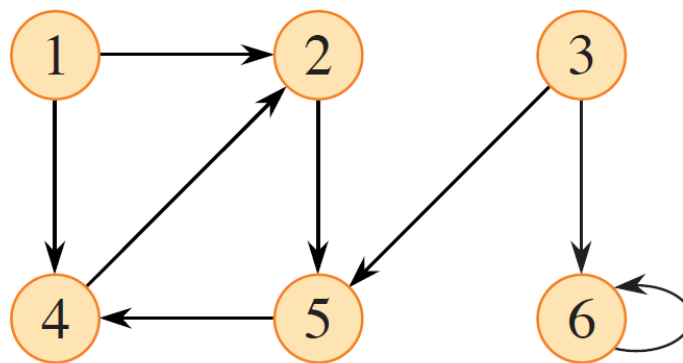
2.5.3 Graf Berarah

Graf berarah (*directed graph* atau *digraph*) adalah graf yang setiap sisinya memiliki arah tertentu. Pada graf berarah, sisi tidak hanya menunjukkan bahwa dua simpul saling terhubung, tetapi juga menunjukkan arah hubungan dari satu simpul menuju simpul lainnya. Secara formal, sisi pada graf berarah dapat dinyatakan sebagai pasangan terurut (u, v) , yang berarti sisi tersebut berawal dari simpul u dan menuju simpul v [3].

Berbeda dengan graf tidak berarah, hubungan pada graf berarah tidak selalu berlaku dua arah. Jika terdapat sisi dari simpul u ke simpul v , maka hal tersebut tidak secara otomatis berarti terdapat sisi dari simpul v ke simpul u . Oleh karena itu, arah sisi menjadi informasi penting dalam memahami pola hubungan antar simpul. Konsep ini banyak digunakan untuk memodelkan hubungan yang memiliki alur, urutan, atau ketergantungan, seperti tautan antar halaman web, aliran data, proses kerja, maupun struktur hierarkis.

Dalam konteks DOM, hubungan antara parent dan child memiliki arah yang jelas, yaitu dari node induk menuju node anak. Misalnya, elemen `<html>` memiliki hubungan ke elemen `<head>` dan `<body>`, kemudian elemen `<body>` dapat memiliki hubungan ke elemen `<div>`, `<p>`, atau elemen lainnya. Arah hubungan tersebut menunjukkan bahwa suatu elemen berada di bawah elemen lain dalam struktur dokumen.

Pemodelan DOM sebagai graf berarah membantu menunjukkan alur hubungan antar elemen secara lebih terstruktur. Dengan menggunakan arah sisi, proses penelusuran dapat dilakukan dari root node menuju node-node turunannya. Hal ini penting dalam analisis struktur halaman web karena urutan dan arah hubungan antar elemen memengaruhi cara DOM dibaca, divisualisasikan, dan ditelusuri menggunakan algoritma traversal seperti DFS.



Gambar 2.3: Contoh graf berarah sederhana (node dan edge)

2.5.4 Representasi DOM sebagai graf

Struktur DOM dapat direpresentasikan sebagai graf dengan cara memetakan setiap elemen HTML sebagai node dan hubungan parent-child sebagai edge. Dalam representasi ini, elemen root seperti `<html>` menjadi node awal, sedangkan elemen-elemen turunannya seperti `<head>`, `<body>`, `<div>`, `<p>`, dan elemen lain menjadi node yang terhubung melalui edge.

Karena hubungan antar elemen dalam DOM memiliki arah dari parent menuju child, struktur DOM lebih sesuai dimodelkan sebagai graf berarah. Setiap edge menunjukkan hubungan hierarkis antara dua elemen. Misalnya, hubungan dari `<body>` ke `<div>` menunjukkan bahwa elemen `<div>` berada di dalam elemen `<body>`. Dengan pemetaan ini, struktur HTML yang semula berbentuk dokumen teks dapat dianalisis sebagai struktur node dan edge.

Representasi DOM sebagai graf mempermudah proses analisis dan visualisasi struktur halaman web. Melalui pendekatan graf, hubungan antar elemen dapat ditelusuri menggunakan algoritma traversal seperti DFS. Selain itu, representasi ini juga mempermudah pembuatan visualisasi meng-

- 1 gunakan Graphviz karena struktur DOM dapat diubah ke dalam format DOT dan ditampilkan
- 2 sebagai diagram graf.

3 2.5.5 Pengertian Traversal Graf

4 Traversal graf adalah proses mengunjungi setiap node dalam graf secara sistematis berdasarkan
5 aturan tertentu. Proses ini dilakukan untuk menelusuri hubungan antar node sehingga seluruh
6 struktur graf dapat diproses tanpa ada node yang terlewat. Dalam kajian algoritma, traversal
7 menjadi langkah penting karena banyak permasalahan graf membutuhkan proses pemeriksaan node
8 dan edge, misalnya untuk mencari jalur, menentukan keterhubungan, menemukan komponen, atau
9 menganalisis struktur hubungan antar objek [3].

10 Pada proses traversal, suatu node biasanya dipilih sebagai titik awal, kemudian algoritma akan
11 menelusuri node-node lain yang terhubung melalui edge. Urutan kunjungan node bergantung
12 pada metode traversal yang digunakan. Beberapa metode traversal yang umum digunakan adalah
13 *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS). BFS menelusuri node berdasarkan
14 tingkat kedekatan dari node awal, sedangkan DFS menelusuri node secara mendalam hingga
15 mencapai cabang terdalam sebelum kembali ke node sebelumnya.

16 Dalam konteks penelitian ini, traversal graf digunakan untuk menelusuri struktur DOM yang
17 telah direpresentasikan sebagai graf. Setiap elemen HTML dipandang sebagai node, sedangkan
18 hubungan parent-child dipandang sebagai edge. Dengan traversal, struktur halaman web dapat
19 dibaca secara teratur dari node awal menuju node-node turunannya. Hal ini membantu proses
20 analisis DOM karena setiap elemen dapat diperiksa sesuai urutan hubungan hierarkisnya.

21 2.5.6 Depth-First Search

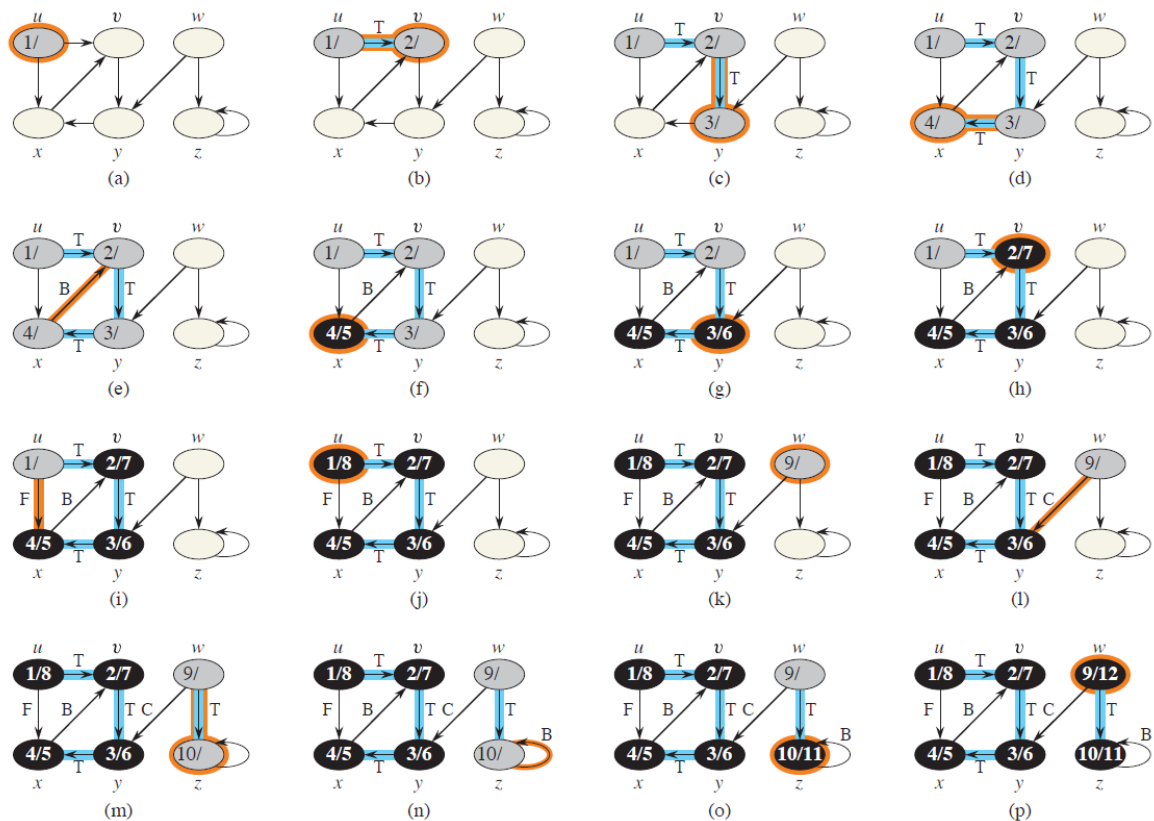
22 *Depth-First Search* (DFS) adalah metode traversal graf yang bekerja dengan menelusuri suatu
23 simpul hingga ke bagian terdalam terlebih dahulu sebelum kembali ke simpul sebelumnya. Sesuai
24 dengan namanya, DFS akan memilih untuk bergerak lebih dalam pada graf selama masih terdapat
25 sisi yang belum dieksplorasi dari simpul yang sedang dikunjungi.

26 Menurut Cormen dkk., DFS mengeksplorasi sisi-sisi yang keluar dari simpul yang paling baru
27 ditemukan dan masih memiliki sisi yang belum diperiksa. Jika seluruh sisi dari simpul tersebut
28 telah dieksplorasi, proses pencarian akan melakukan *backtracking*, yaitu kembali ke simpul asal
29 tempat simpul tersebut ditemukan untuk melanjutkan eksplorasi sisi lain yang belum dikunjungi [3].
30 Proses ini berlanjut hingga seluruh simpul yang dapat dicapai dari simpul awal berhasil ditemukan.
31 Apabila masih terdapat simpul lain yang belum ditemukan, DFS akan memilih simpul tersebut
32 sebagai sumber baru dan mengulangi proses pencarian sampai seluruh simpul dalam graf telah
33 diperiksa.

34 Dalam prosesnya, DFS juga mencatat hubungan pendahulu (*predecessor*) dari setiap simpul
35 yang ditemukan. Ketika suatu simpul baru ditemukan dari simpul lain, hubungan tersebut dicatat
36 sebagai bagian dari struktur hasil pencarian. Pada graf yang tidak seluruh simpulnya terhubung dari
37 satu sumber, DFS dapat menghasilkan beberapa pohon pencarian, sehingga hasil keseluruhannya
38 disebut sebagai *depth-first forest*. Sisi-sisi yang membentuk hubungan pendahulu tersebut dikenal
39 sebagai *tree edges* [3].

DFS menggunakan penandaan warna untuk menunjukkan status setiap simpul selama proses traversal. Simpul yang belum dikunjungi diberi warna putih, simpul yang sedang diproses diberi warna abu-abu, dan simpul yang seluruh sisi keluarannya telah selesai diperiksa diberi warna hitam. Selain itu, DFS juga mencatat dua waktu pada setiap simpul, yaitu waktu penemuan (*discovery time*) ketika simpul pertama kali dikunjungi dan waktu penyelesaian (*finishing time*) ketika seluruh daftar ketetanggaan simpul tersebut telah selesai diperiksa. Informasi ini berguna untuk memahami urutan traversal dan struktur hubungan antar simpul dalam graf [3].

Metode ini sangat cocok untuk struktur berbentuk tree seperti DOM karena mengikuti pola parent-child secara mendalam. Pada konteks DOM, DFS dapat digunakan untuk menelusuri elemen dari node induk ke node anak secara bertahap hingga mencapai elemen terdalam, kemudian kembali ke node sebelumnya untuk melanjutkan penelusuran ke cabang lain. Oleh karena itu, DFS sering digunakan untuk eksplorasi struktur hierarkis dan graf [3].



Gambar 2.4: Ilustrasi proses DFS pada graf berarah. Sisi-sisi diklasifikasikan sesuai urutan penjelajahannya: sisi pohon diberi label T, sisi belakang B, sisi depan F, dan sisi silang C.

2.6 Graphviz

2.6.1 Pengertian Graphviz

Graphviz adalah perangkat lunak visualisasi grafik *open source*. Visualisasi grafik adalah cara untuk merepresentasikan informasi struktural sebagai diagram grafik abstrak dan jaringan. Teknologi ini memiliki aplikasi penting dalam jaringan, bioinformatika, rekayasa perangkat lunak, desain basis data dan web, pembelajaran mesin, dan antarmuka visual untuk domain teknis lainnya[4].

DAFTAR REFERENSI

- [1] Marini, J. (2002) *The Document Object Model*. McGraw-Hill/Osborne Media, America.
- [2] Duckket, J. (2011) *HTML and CSS, Design and Build Websites*. John Wiley and Sons, Inc, Canada.
- [3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., dan Stein, C. (2022) *Introduction to Algorithms*, 4th edition. The MIT Press, America.
- [4] Ellson, J., Gansner, E., Hu, Y., North, S., Jacobsson, M., Fernandez, M., dan Hansen, M. (2024) Graphviz. <https://graphviz.org/>. 28 April 2026.

LAMPIRAN A

KODE PROGRAM

Kode A.1: MyCode.c

```
1 // This does not make algorithmic sense,
2 // but it shows off significant programming characters.
3
4 #include<stdio.h>
5
6 void myFunction( int input, float* output ) {
7     switch ( array[i] ) {
8         case 1: // This is silly code
9             if ( a >= 0 || b <= 3 && c != x )
10                 *output += 0.005 + 20050;
11             char = 'g';
12             b = 2^n + ~right_size - leftSize * MAX_SIZE;
13             c = (--aaa + &daa) / (bbb++ - ccc % 2 );
14             strcpy(a,"hello_$@?");
15         }
16         count = ~mask | 0x00FF00AA;
17     }
18 }
19
20 // Fonts for Displaying Program Code in LATEX
21 // Adrian P. Robson, nepsweb.co.uk
22 // 8 October 2012
23 // http://nepsweb.co.uk/docs/progfonts.pdf
```

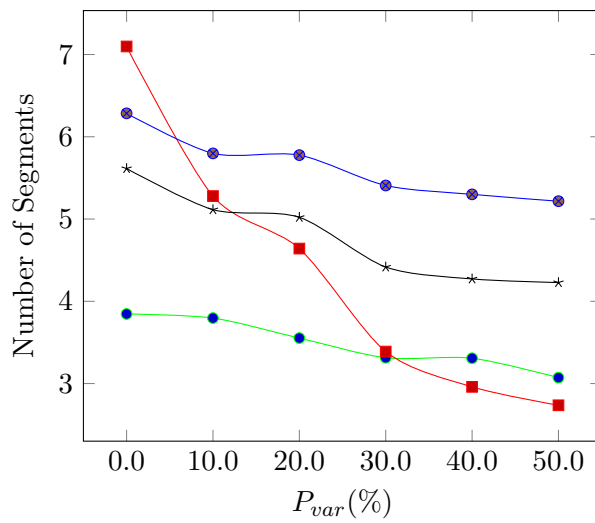
Kode A.2: MyCode.java

```
1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.HashSet;
4
5 //class for set of vertices close to furthest edge
6 public class MyFurSet {
7     protected int id; //id of the set
8     protected MyEdge FurthestEdge; //the furthest edge
9     protected HashSet<MyVertex> set; //set of vertices close to furthest edge
10    protected ArrayList<ArrayList<Integer>> ordered; //list of all vertices in the set for each trajectory
11    protected ArrayList<Integer> closeID; //store the ID of all vertices
12    protected ArrayList<Double> closeDist; //store the distance of all vertices
13    protected int totaltrj; //total trajectories in the set
14
15    /*
16     * Constructor
17     * @param id : id of the set
18     * @param totaltrj : total number of trajectories in the set
19     * @param FurthestEdge : the furthest edge
20     */
21    public MyFurSet(int id,int totaltrj,MyEdge FurthestEdge) {
22        this.id = id;
23        this.totaltrj = totaltrj;
24        this.FurthestEdge = FurthestEdge;
25        set = new HashSet<MyVertex>();
26        ordered = new ArrayList<ArrayList<Integer>>();
27        for (int i=0;i<totaltrj;i++) ordered.add(new ArrayList<Integer>());
28        closeID = new ArrayList<Integer>(totaltrj);
29        closeDist = new ArrayList<Double>(totaltrj);
30        for (int i = 0;i <totaltrj;i++) {
31            closeID.add(-1);
32            closeDist.add(Double.MAX_VALUE);
33        }
34    }
35
36 }
```

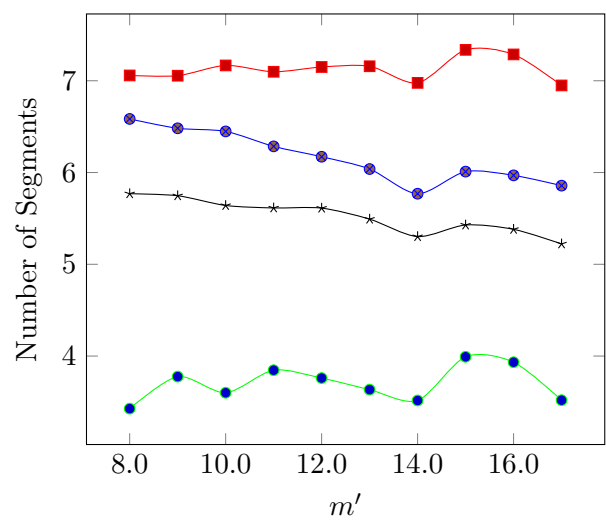

LAMPIRAN B

HASIL EKSPERIMEN

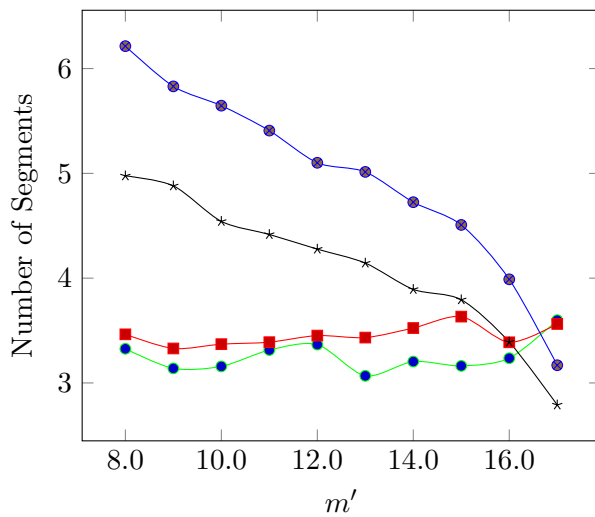
Hasil eksperimen berikut dibuat dengan menggunakan TIKZPICTURE (bukan hasil excel yg diubah ke file bitmap). Sangat berguna jika ingin menampilkan tabel (yang kuantitasnya sangat banyak) yang datanya dihasilkan dari program komputer.



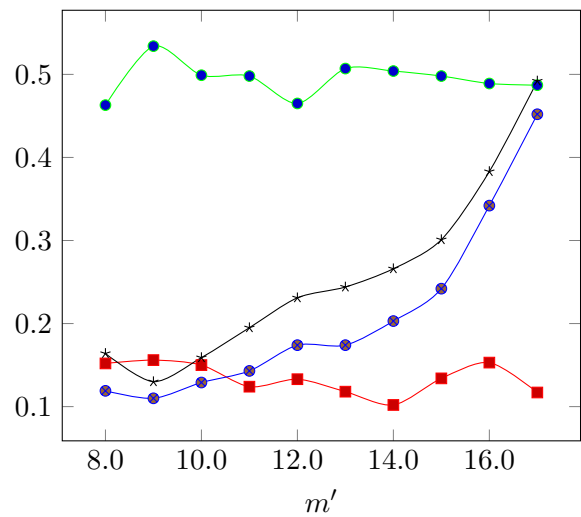
Gambar B.1: Hasil 1



Gambar B.2: Hasil 2



Gambar B.3: Hasil 3



Gambar B.4: Hasil 4